PROJECT DOCUMENTATION

Title: Chatbot using AWS Bedrock

1. Introduction

This project presents the development of a web-based AI chatbot powered by **Meta's LLaMA 3 model** deployed through **AWS Bedrock**. The system enables users to interact with a large language model in real time via a simple and interactive interface built using Streamlit.

The application is designed with a focus on **cost-efficient inference**, secure credential handling, and controlled usage. It demonstrates how cloud-based LLM services can be integrated into lightweight applications for conversational AI tasks.

2. Objectives

- To develop a real-time chatbot using AWS Bedrock
- To integrate Meta's LLaMA 3 model for text generation
- To design a user-friendly interface using Streamlit
- To implement secure handling of AWS credentials
- To control API usage and cost through request limits
- To demonstrate cloud-based LLM integration

3. Technologies and Libraries Used

3.1 Technologies

- Python: Core programming language
- Streamlit: Web-based user interface
- AWS Bedrock: Cloud platform for LLM inference
- LLaMA 3 (8B Instruct): Language model used for responses

3.2 Libraries

- streamlit: UI rendering and interaction
- boto3: AWS SDK for Python
- python-dotenv: Environment variable management
- json: Request and response handling
- os: Environment access

4. System Architecture

4.1 Workflow

User Input → Streamlit Interface → AWS Credential Setup → Bedrock Client → Prompt Formatting → LLaMA 3 Model (AWS Bedrock) → Response Processing → Output Display

4.2 Component Description

User Interface (Streamlit)
Provides input fields for AWS credentials and user queries. Displays chat history dynamically.

AWS Configuration
Allows users to input Access Key, Secret Key, and region. These credentials are used to establish a session with AWS Bedrock.

Bedrock Client Initialization
Uses boto3 to create a Bedrock runtime client for invoking the LLaMA 3 model.

Prompt Construction
Formats user input into a structured prompt compatible with the LLaMA instruction format.

Model Invocation
Sends a request to AWS Bedrock using:

- Model ID: meta.llama3-8b-instruct-v1:0
- Parameters: temperature, top_p, max_gen_len

Response Processing
Parses the returned JSON response and extracts generated text. Removes unnecessary tokens and formatting.

Session Management
Stores chat history and message count using Streamlit session state.

5. Features

- Real-time chatbot interaction using AWS Bedrock
- Integration with LLaMA 3 (8B Instruct model)
- Secure AWS credential input via UI
- Chat history tracking within session
- Cost control using message limit
- Clean and minimal user interface

6. Code Implementation

```
import streamlit as st
import boto3
import json
import os
from dotenv import load_dotenv

load_dotenv()

# Page config
st.set_page_config(page_title="LLaMA 3 Chatbot", page_icon="🤖", layout="wide")

st.title("🤖 LLaMA 3 Chatbot")
st.caption("Powered by AWS Bedrock (Cost Optimized)")

# Session state
if "chat_history" not in st.session_state:
    st.session_state.chat_history = []

if "bedrock_client" not in st.session_state:
    st.session_state.bedrock_client = None

if "message_count" not in st.session_state:
    st.session_state.message_count = 0



# Sidebar
with st.sidebar:
    st.header("⚙ AWS Configuration")

    aws_access_key = st.text_input(
        "AWS Access Key ID",
```

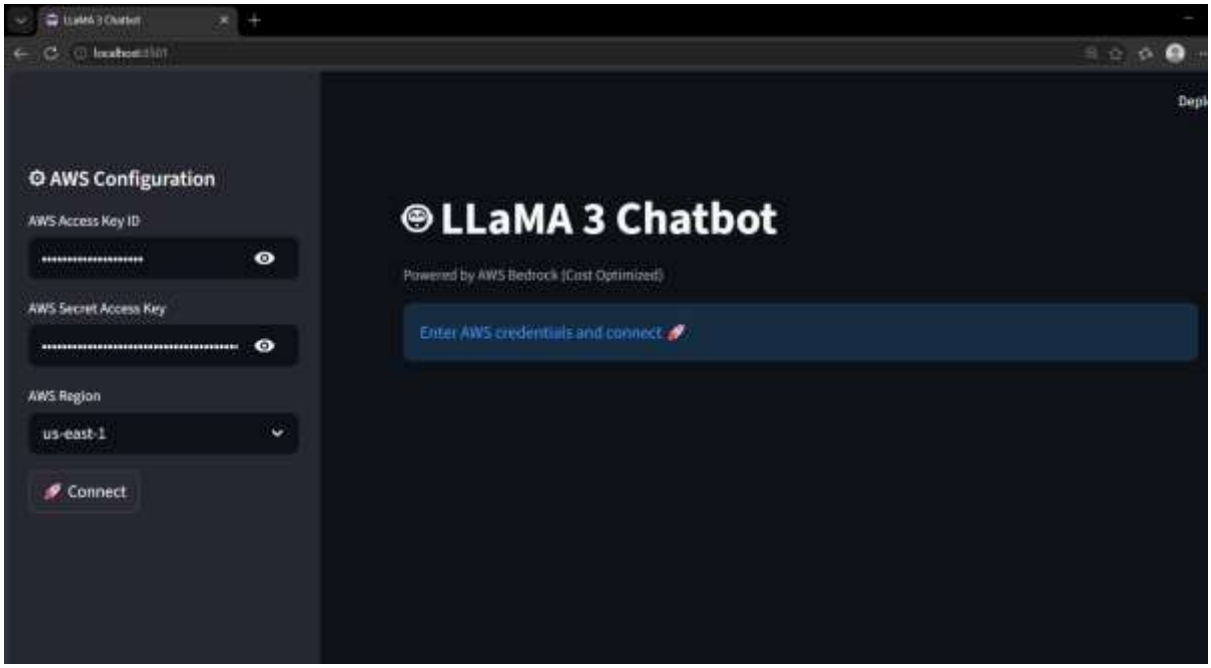
```
        value=os.getenv("AWS_ACCESS_KEY_ID", ""),
        type="password"
    )

    aws_secret_key = st.text_input(
        "AWS Secret Access Key",
        value=os.getenv("AWS_SECRET_ACCESS_KEY", ""),
        type="password"
    )

    region = st.selectbox("AWS Region", ["us-east-1"])

    connect = st.button("🚀 Connect")


# Connect
if connect:
    try:
        session = boto3.Session(
            aws_access_key_id=aws_access_key,
            aws_secret_access_key=aws_secret_key,
            region_name=region
        )
        st.session_state.bedrock_client = session.client("bedrock-runtime")
        st.sidebar.success("✅ Connected")
    except Exception as e:
        st.sidebar.error(f"❌ {e}")


# Chat UI
if st.session_state.bedrock_client:
```

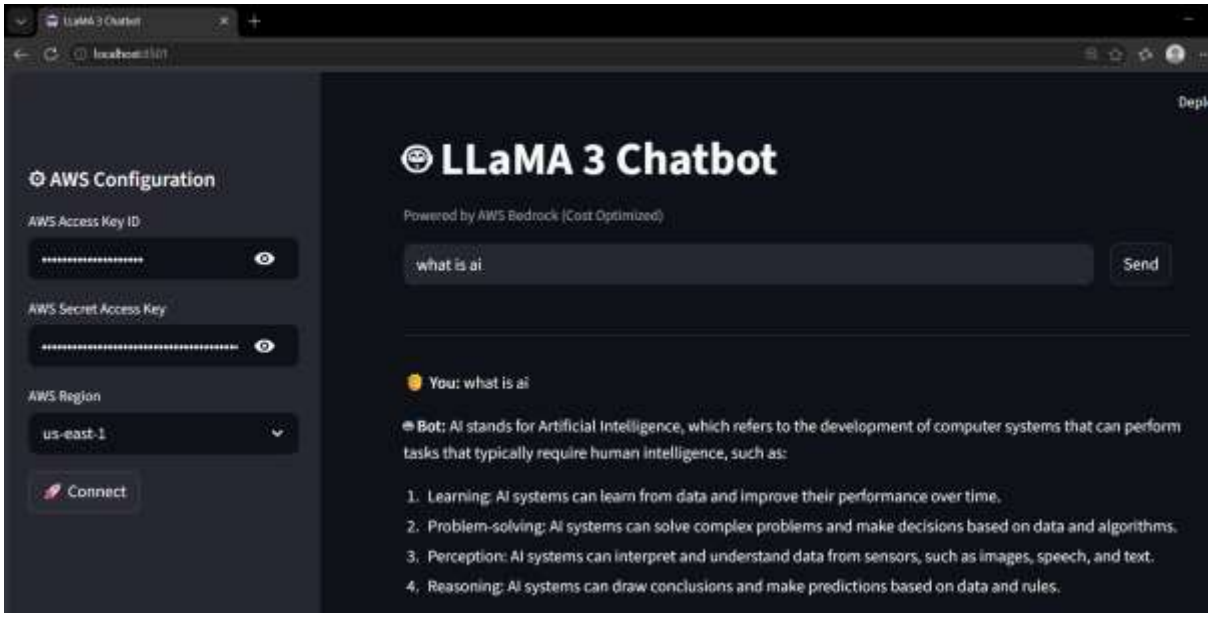
```
# 🚨 HARD LIMIT (₹10 protection)
if st.session_state.message_count >= 25:
    st.warning("🚫 Demo limit reached (cost protection enabled)")
    st.stop()

col1, col2 = st.columns([8, 1])

with col1:
    user_input = st.text_input("Message", placeholder="Type...", label_visibility="collapsed")

with col2:
    send = st.button("Send")

if send and user_input.strip():

    st.session_state.chat_history.append({
        "role": "user",
        "content": user_input
    })

    # ✅ FIXED PROMPT (ONLY CURRENT INPUT)
    prompt = f"<|user|>\n{user_input}\n<|assistant|>\n"

    body = {
        "prompt": prompt,
        "temperature": 0.5,
        "top_p": 0.8,
        "max_gen_len": 300
    }
```

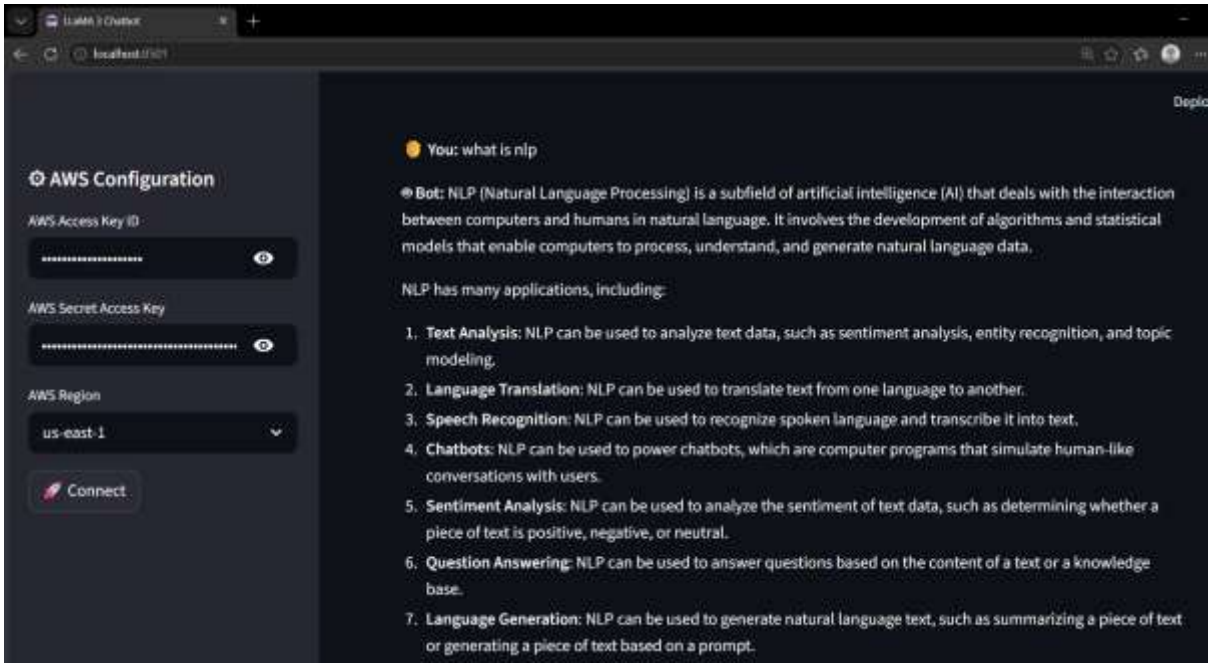
```
    try:
        response = st.session_state.bedrock_client.invoke_model(
            modelId="meta.llama3-8b-instruct-v1:0",
            body=json.dumps(body),
            contentType="application/json",
            accept="application/json"
        )

        response_body = json.loads(response["body"].read())

        output_text = response_body.get("generation", "No response")

        # Clean output
        if "Assistant:" in output_text:
            output_text = output_text.split("Assistant:")[-1].strip()

        if "<|assistant|>" in output_text:
            output_text = output_text.split("<|assistant|>")[-1].strip()

        st.session_state.chat_history.append({
            "role": "assistant",
            "content": output_text
        })

        # count messages
        st.session_state.message_count += 1

    except Exception as e:
        st.error(f"❌ {e}")

st.markdown("---")
```

```
    for turn in st.session_state.chat_history:
        if turn["role"] == "user":
            st.markdown(f"👤 **You:** {turn['content']}")
        else:
            st.markdown(f"🤖 **Bot:** {turn['content']}")

else:
    st.info("Enter AWS credentials and connect 🚀")
```

7. Output







8. Conclusion

This project demonstrates how AWS Bedrock can be leveraged to build scalable and efficient AI chatbot applications. By integrating LLaMA 3 with a Streamlit interface, the system provides a practical example of cloud-based LLM usage with cost control and session management. The modular design allows easy extension for more advanced AI-driven applications.